

C++ QUICK REFERENCE

C++98

Preprocessor

```
// Comment to end of line
/* Multi-line comment */
#include <stdio.h> // Insert standard header file
#include "myfile.h" // Insert file in current directory
#define X some text // Replace X with some text
#define F(a,b) a+b // Replace F(1,2) with 1+2
#define X \
some text // Multiline definition
#undef X // Remove definition
#if defined(X) // Conditional compilation (#ifdef X)
#else // Optional (#ifndef X or #if !defined(X))
#endif // Required after #if, #ifdef
```

Data Types

Data Type	Description
bool	boolean (true or false)
char	character ('a', 'b', etc.)
char[]	character array (C-style string if null terminated)
string	C++ string (from the STL)
int	Integer (1, 2, -1, 1000, etc.)
long int	long integer
float	single precision floating point
double	double precision floating point

These are the most commonly used types: this is not a complete list.

Operators

The most commonly used operators in order of precedence:

- ➊ ++ (post-increment), -- (post-decrement)
- ➋ !(not), ++ (pre-increment), -- (pre-decrement)
- ➌ *, %(modulus)
- ➍ +-
- ➎ <, <=, >, >=

- ➏ == (equal-to), != (not-equal-to)
- ➐ && (and)
- ➑ || (or)
- ➒ = (assignment), *, /, %=, +=, -=

Looping

while Loop

```
while (expression)
statement;
```

Example:

```
while (x < 100)
cout << x++ << endl;
```

```
while (expression){
statement;
statement;
}
```

Example:

```
while (x < 100){
cout << x << endl;
x++;
}
```

do-while Loop

```
do
statement;
while (expression);
```

Example:

```
do
cout << x++ << endl;
while (x < 100);
```

```
do{
statement;
statement;
} while (expression);
```

Example:

```
do {
cout << x << endl;
x++;
} while (x < 100);
```

For Loop

```
for (initialization; test; update)
statement;
```

```
for (initialization; test; update){
statement;
statement;
}
```


C++ QUICK REFERENCE

OOP, Templates, Exceptions

Example:

Inheritance

```
class Base {
public:
    virtual ~Base() {}
    virtual void f() = 0;
};
class Derived : public Base {
public:
    void f() { /* implementation */ }
};
```

Polymorphism

```
Base* b = new Derived();
b->f();
delete b; // calls Derived::~Derived then
          Base::~Base
```

Operator Overloading

```
struct Vec { int x,y; };
Vec operator+(const Vec& a, const Vec& b){
    Vec r; r.x = a.x + b.x; r.y = a.y + b.y;
    return r; }
std::ostream& operator<<(std::ostream& os,
```

```
const Vec& v){ return os << v.x << ", "
<< v.y; }
```

Exceptions

```
#include <stdexcept>
#include <iostream>
int divide(int a,int b){ if(b==0) throw
    std::runtime_error("divide by zero");
    return a/b; }
try { int r = divide(4,0); } catch(const
    std::exception &e) { std::cout << e.what
    (); }
```

Function Templates

```
template <class T>
T maxT(T a,T b){ return (a>b)?a:b; }
int m = maxT(3,5);
```

Class Templates

```
template <class T>
struct Pair{ T first, second; };
Pair<int> p; p.first = 1; p.second = 2;
```


C++ QUICK REFERENCE

Functions and Classes

Passing Parameters by Value

```
return_type function(type p1)
```

Variable is passed into the function but changes to p1 are not passed back.

Passing Parameters by Reference

```
return_type function(type &p1)
```

Variable is passed into the function and changes to p1 are passed back.

Default Parameter Values

```
return_type function(type p1=val)
```

val is used as the value of p1 if the function is called without a parameter.

Struct/Classes

Declaration

```
class classname {
public:
```

```
    classname (params);
    ~classname();
    type member1;
    type member2;
protected:
    type member3;
private:
    type member4;
};
```

```
Example:
class Square {
public:
    Square();
    Square (float w);
    void setWidth(float w);
    float getArea();
private:
    float width;
};
```

- public members are accessible from anywhere the class is visible.
- private members are only accessible from the same class or a friend (function or class).
- protected members are accessible from the

same class, derived classes, or a friend (function or class).

- constructors may be overloaded just like any other function. You can define two or more constructors as long as each constructor has a different parameter list.

Definition of Member Functions

```
return_type classname::functionName (params) {
    statements;
}
```

```
Examples:
Square::Square() {
    width = 0;
}
void Square::setWidth(float w) {
    if (w >= 0)
        width = w;
    else
        exit(-1);
}
float Square::getArea() {
```

```
    return width * width;
}
```

Definition of Instances

```
classname varName;
classname* ptrName;
```

```
Example:
Square s1();
Square s2(3.5);
Square* sPtr;
sPtr = new Square (1.8);
```

Accessing Members

```
varName.member = val;
varName.member();
ptrName->member = val;
ptrName->member();
```

```
Example:
s1.setWidth(1.5);
cout << s1.getArea();
cout << sPtr->getArea();
```


C++ QUICK REFERENCE

OOP, Templates, Exceptions

OOP Principles

Encapsulation: Hide internal implementation details by using access specifiers (public, private, protected) to control access to member variables and functions.

Abstraction: Define interfaces that represent only essential features while hiding complexity. Use abstract classes with pure virtual functions.

Inheritance

Inheritance allows a new class to be based on an existing class. The new class inherits all the member variables and functions (except the constructors and destructor) of the class it is based on.

```
Example:
class Student
{
public:
    Student (string n, string id);
    void print();
protected:
    string name;
    string netID;
};
class GradStudent : public Student
{
public:
    GradStudent(string n, string id, string prev);
    void print();
protected:
    string prevDegree;
};
```

Visibility of Members after Inheritance

Inheritance Specification	Access Specifier in Base Class	Access Specifier in Derived Class
private	private	protected
protected	protected	protected
public	public	public

Polymorphism

Compile-time (Static) Polymorphism: Function overloading and templates.

Runtime (Dynamic) Polymorphism: Use virtual functions to allow derived classes to override base class methods. Write code that works with base class pointers/references to handle derived classes dynamically.

```
Example:
class Shape // Abstract class
{
public:
    virtual void draw() = 0; // pure virtual
    virtual double area() = 0;
    virtual ~Shape() {}
};

class Circle : public Shape
{
public:
    void draw() { /* implementation */ }
    double area() { return 3.14 * r * r; }
private:
    double r;
};

Shape* shapes[10];
shapes[0] = new Circle();
shapes[0]->draw(); // calls Circle::draw()
```

Operator Overloading

C++ allows you to define how standard operators (+, -, *, etc.) work with classes that you write. For example, to use the operator + with your class, you would write a function named operator+ for your class. Example: Prototype for a function that overloads + for the Square class:

```
Square operator+ (const Square &);
```

If the object that receives the function call is not an instance of a class that you wrote, write the function as a friend of your class. This is standard practice for overloading « and ».

Example: Prototype for a function that overloads « for the Square class:

```
friend ostream & operator<< (ostream &, const Square &);
```

Make sure the return type of the overloaded function matches what C++ programmers expect. The return type of relational operators (<, >, ==, etc.) should be bool, the return type of « should be ostream &, etc.

Namespaces

```
namespace N { class T {}; }
N::T t;
using namespace N;
```

Exceptions

```
Example:
try
{
    // code here calls functions that might
    // throw exceptions
    quotient = divide (num1, num2);

    // or this code might test and throw
    // exceptions directly
    if (num3 < 0)
        throw -1; // exception to be thrown
                    can
                    // be a value or an object
}
catch (int)
{
    cout << "num3 can not be negative!";
    exit(-1);
}
catch (char* exceptionString)
{
    cout << exceptionString;
    exit(-2);
}
// add more catch blocks as needed
```

Function Templates

```
Example:
template <class T>
T getMax(T a, T b)
{
    if (a > b)
        return a;
    else
        return b;
```

```
}
```

```
Example calls:
int a = 9, b = 2, c;
c = getMax(a, b);
float f = 5.3, g = 9.7, h;
h = getMax(f, g);
```

Class Templates

```
Example:
template <class T>
class Point
{
public:
    Point(T x, T y);
    void print();
    double distance (Point<T> p);
private:
    T x;
    T y;
};
```

```
Examples:
Point<int> p1(3, 2);
Point<float> p2(3.5, 2.5);
p1.print();
p2.print();
```

Suggested Websites

- C++ Reference: <http://www.cppreference.com/>
- C++ Tutorial: <http://www.cplusplus.com/doc/tutorial/>
- <http://www.informit.com/guides/guide.aspx?g=cplusplus>
- <http://www.sparknotes.com/cs/>
- C++ Examples: <http://www.fredosaurus.com/notes-cpp/>
- Gaddis Textbook: http://media.pearsoncmg.com/aw/aw_gaddis_sowcso_6/
- Video Notes Source Code: <ftp://ftp.aw.com/cseng/authors/gaddis/CCSos> (5th edition)